

Job Shop Scheduling

These notes are based off the excellent resources provided by [Google OR-Tools](#).

1. Constraint Programming

Constraint Satisfaction Problem (CSP)

The Constraint Satisfaction Problem (CSP) is defined by:

- a set of variables $X = \{x_1, x_2, \dots, x_n\}$
- each with a domain $D = \{d_1, d_2, \dots, d_n\}$ of possible values
- and a set of constraints $C = \{c_1, c_2, \dots, c_m\}$
 - where each constraint $c_i = (S_i, R_i)$ involves a subset of variables, $S_i = (x_{i_1}, x_{i_2}, \dots, x_{i_k})$ (scope), and specifies the allowable combinations of values for those variables $R \subseteq d_{i_1} \times \dots \times d_{i_k}$ (relation).

The goal is to find an assignment of values to the variables from their respective domains such that all constraints are satisfied simultaneously.

Formally, a solution to the CSP is an assignment $A = \{x_1 = v_1, x_2 = v_2, \dots, x_n = v_n\}$ such that:

- Each variable x_i is assigned a value v_i from its domain D_i .
- For each constraint c_i involving variables $(x_{i_1}, x_{i_2}, \dots, x_{i_k})$, the combination of values $(v_{i_1}, v_{i_2}, \dots, v_{i_k})$ satisfies the constraint C_i .

The CSP can be represented as a tuple $\langle X, D, C \rangle$, where:

- X is the set of variables.
- $D = \{d_1, d_2, \dots, d_n\}$ is the set of domains for the variables.
- C is the set of constraints.

Note: An objective is optional. For example, sudoku is a CSP and has no objective! You just need to find a solution that satisfies the constraints

Example Constraint

Consider the constraint:

$$\begin{aligned}x + y &= z \\ x, y, z &\in \{0, 1\}\end{aligned}$$

Then the constraint is given by $c = ((x, y, z), \{(0, 0, 0), (1, 0, 1), (0, 1, 1)\})$

Example problem

$$\begin{aligned} &\text{maximise} && 2x + 2y + 3z \\ &\text{subject to:} && 3x - 5y + 7z \leq 45 \\ &&& 2y - 6z \leq 37 \\ &&& x, y, z \geq 0 \\ &&& x, y, z \in \mathbb{Z} \end{aligned}$$

You will also recognise this as an integer programming problem.

You already know how to solve this using Pyomo!

You can also see how to solve this using [Google OR-Tools CP-SAT solver](#). Note that this is a different technique and solver to the one we've been using in Pyomo (GLPK).

Constraint Programming vs Mathematical Programming

You should look at the following to see the differences - [IBM Decision Optimisation - MP vs CP](#).

Perhaps the most fundamental difference is this:

	Linear (MILP/Integer) Programming	Quadratic Programming	Constraint Programming
Modelling Limitations	Restricted to linear problems	Restricted to quadratic problems	Restricted to discrete problems

2. Cryptarithmic

A cryptarithmic puzzle is a mathematical exercise where:

- The digits of some numbers are represented by letters (or symbols).
- Each letter represents a unique digit.
- The goal is to find the digits such that a given mathematical equation is verified

$$\begin{array}{r} \text{OR} \\ + \text{IS} \\ + \text{FUN} \\ \hline = \text{TRUE} \end{array}$$

This has 96 possible solutions. You should confirm that the two below are solutions!

$$\begin{array}{r} 20 \\ + 75 \\ + 948 \\ \hline = 1043 \end{array}$$

$$\begin{array}{r} 60 \\ + 37 \\ + 928 \\ \hline = 1025 \end{array}$$

Converting to a CSP Model

Variables

- Based on the equation $\text{OR} + \text{IS} + \text{FUN} = \text{TRUE}$ there are 8 variables.
- The domain of each variable is $\{0, 1, \dots, 9\}$.

Constraints:

- $\text{OR} + \text{IS} + \text{FUN} = \text{TRUE}$
- Each of the 8 letters is unique. Note this is called the all different constraint
- O, I, F and T cannot be 0 (as we don't start numbers in decimal with a leading 0).

Note: There is no objective!

Solution

You can find the how to implement the solution to a similar problem on the [Google OR-Tools site](#).

3. What is Job Shop Scheduling

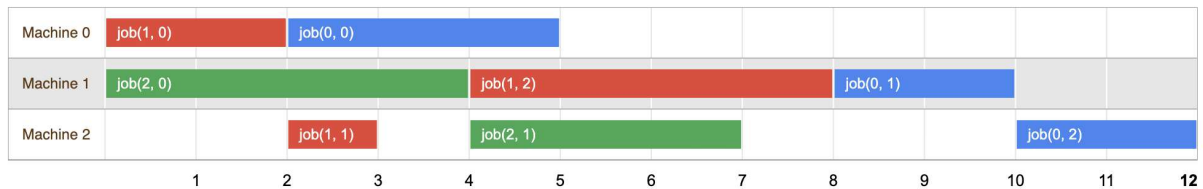
It is quite common in manufacturing to have machines that perform a given task for a range of products.

A complete job may have a sequence of tasks that require the use of different machines, each with their own processing time.

Below we have 3 jobs and 3 machines. Each job requires a sequence of tasks using the machines.

Jobs	Tasks	Sequence on Machines	Process Time on Machines
J_0	t_{01}, t_{02}, t_{03}	M_0, M_1, M_2	3,2,2
J_1	t_{11}, t_{12}, t_{13}	M_0, M_2, M_1	2,1,4
J_2	t_{21}, t_{22}, t_{23}	M_1, M_2	4,3

Here is an example schedule which is valid.



The question is how do we best schedule these jobs on the machines?

Note that we can also express this problem as,

Job 0 = [(0, 3), (1, 2), (2, 2)]

Job 1 = [(0, 2), (2, 1), (1, 4)]

Job 2 = [(1, 4), (2, 3)]

here the tuple of the form (m,p) represents:

- m = machine number
- p = processing time on m.

This is useful when writing code to solve these problems.

History

1. Early Development (1950s-1960s):

- The job shop scheduling problem emerged in the context of manufacturing operations during the 1950s and 1960s.
- Researchers began formalizing scheduling problems and exploring mathematical approaches to solve them.

2. Algorithmic Development (1970s-1980s):

- In the 1970s and 1980s, significant progress was made in developing algorithms to solve job shop scheduling problems.
- Branch and bound algorithms were explored, along with mathematical programming techniques such as linear and integer programming.

3. Heuristic Methods (1980s-Present):

- From the 1980s onwards, heuristic methods gained popularity for solving job shop scheduling problems.
- Heuristic algorithms, including genetic algorithms, simulated annealing, tabu search, and ant colony optimization, were developed to find approximate solutions efficiently.

4. Exact Algorithms (1990s-Present):

- In the 1990s and continuing into the present, research focused on developing exact algorithms for job shop scheduling.
- Exact methods, such as branch and bound with dynamic programming and constraint programming, aim to find optimal solutions by exhaustively searching through all possible schedules.

5. Hybrid Approaches and Applications (Present):

- Current research in job shop scheduling emphasizes hybrid approaches that combine different algorithms and techniques to improve solution quality and computational efficiency.
- Job shop scheduling has widespread applications across various industries, including manufacturing, transportation, healthcare, and services, where efficient resource scheduling is crucial for optimizing productivity and minimising costs.

Formal Definition

The job shop scheduling problem involves scheduling a set of n jobs on a set of m machines.

- Each job consists of a fixed sequence of tasks
- Each task must be processed on a specific machine.
- The **goal** is to determine the **optimal schedule** for executing these predetermined sequences of tasks on the available machines in order to **minimise the makespan**.

Given:

- n jobs: $(J_1, J_2, \dots, J_n)$
- m machines: $(M_1, M_2, \dots, M_m)$
- Each job J_i has a sequence of tasks $J_i = (t_{i1}, t_{i2}, \dots, t_{ik})$
- Each task $t_{ij} = (M_k, p_{ijk})$ where p_{ijk} is the processing time for task ij on machine k .

The objective is to find:

- A schedule (S_{ijk}) indicating the start time of task t_{ij} on machine M_k

Such that:

- **Precedence**: Each task is scheduled to start only after all preceding tasks for the same job have been completed (i.e. maintain the job sequence!).
 - $S_{ijs} + p_{ijs} \leq S_{ikt}$ (For job i the start time for consecutive tasks ij and ik is such that task ij plus its processing time on machine s is less than the start time for task ik on machine t .)
- **No Overlap**: A machine can only do one task at a time and must therefore finish a task before starting one.
 - $S_{ijs} + p_{ijs} \leq S_{kls}$ or $S_{kls} + p_{kls} \leq S_{ijs}$ - Jobs on the same machine s cannot overlap. Note this is two disjunctive (or) constraints, i.e. one must hold!
- The total makespan, $(C_{\max})$ is minimised.
 - i.e. minimise the length of time from the earliest start time of the jobs to the latest end time.

Solving with a Sat Solver

You can find out how to solve this via [Google OR-Tools](#)